

Documentación de Formalismos

Corrector de Concordancia de Género y Número en Español

Procesamiento de Lenguaje Natural — Universidad del Valle · Semestre 2026-1

Cristhian Leonardo Albarracín Zapata · Código: 1968253

1. Autómata Finito Determinista (DFA) — Tokenización

El sistema implementa un DFA que procesa la frase de entrada carácter a carácter y emite tokens cada vez que detecta el fin de una palabra. La representación formal es la 5-tupla:

Q	{ q0, q1, q2 }
Σ	letras del español (a-z, á, é, í, ó, ú, ü, ñ, A-Z y sus acentuadas)
q0	q0 (estado inicial)
F	{ q1 } (q1 indica que se está acumulando una palabra válida)

Función de transición δ :

Estado	Entrada = letra	Entrada = otro carácter
q0 (inicio)	q1	q2
q1 (en palabra)	q1	q2 (emite token)
q2 (separador)	q1	q2

La transición $q1 \rightarrow q2$ es el evento que dispara la emisión del token acumulado en el buffer. Implementado en la clase DFA mediante los métodos `_delta()` y `tokenizar()`.

2. Gramática de Contexto Libre (CFG)

La gramática que define la estructura sintáctica del sistema se especifica formalmente como $G = (V, T, P, S)$:

V	{ O, SN, SV, Det, N, Adj, V } — símbolos no terminales
T	{ el, la, los, las, un, una, unos, unas, gato, niña, perro, ... } — léxico
S	O — símbolo inicial

Producciones P:

Producción	Descripción
$O \rightarrow SN\ SV$	Oración completa
$O \rightarrow SN$	Sintagma nominal independiente
$SN \rightarrow Det\ N\ Adj^*$	Sintagma nominal con determinante
$SN \rightarrow N\ Adj^*$	Sintagma nominal sin determinante
$SV \rightarrow V\ SN$	Sintagma verbal con objeto
$SV \rightarrow V$	Sintagma verbal sin objeto
$Det \rightarrow el\ \ la\ \ los\ \ las\ \ un\ \ una\ \ \dots$	Determinantes
$N \rightarrow gato\ \ niña\ \ perro\ \ casa\ \ \dots$	Sustantivos
$Adj \rightarrow alto\ \ alta\ \ negro\ \ gris\ \ \dots$	Adjetivos

V → corre ladra duerme come ...	Verbos
---	--------

3. Parser Descendente Recursivo

Se implementó un parser descendente recursivo (top-down) que recorre los tokens de izquierda a derecha intentando derivar la oración desde el símbolo inicial O. Las funciones parse_sn(), parse_sv() y parse_o() corresponden directamente a las producciones de la CFG. El parser construye simultáneamente el árbol de derivación mediante la clase Nodo(etiqueta, hijos).

4. DCG / Unificación con DAGs

Cada palabra del léxico está representada como un DAG (Directed Acyclic Graph) con rasgos morfológicos. La función unificar(dag1, dag2) combina dos DAGs: retorna el DAG fusionado si los rasgos son compatibles, o None si hay conflicto. Esta operación es el mecanismo central de verificación de concordancia.

Ejemplo de DAGs y unificación:

Palabra	DAG de rasgos
"la"	{ cat: 'det', gen: 'fem', num: 'sing' }
"niña"	{ cat: 'n', gen: 'fem', num: 'sing' }
"alto"	{ cat: 'adj', gen: 'masc', num: 'sing' }

Expresión	Resultado
unificar({gen:"fem", num:"sing"}, {gen:"fem", num:"sing"})	OK — "la niña" concuerda en género y número.
unificar({gen:"fem", num:"sing"}, {gen:"masc", num:"sing"})	None — "la niña alto" falla: género fem ≠ masc.

5. Integración de las herramientas

Las seis herramientas se encadenan en el siguiente flujo para procesar cada frase de entrada:

Paso	Herramienta	Entrada	Salida
1	DFA	Frase en texto	Lista de tokens
2	CFG + Parser	Lista de tokens	Árbol de derivación + DAGs de SN
3	DCG/Unificación	DAGs det/sust/adj	OK o None por conflicto
4	Ambigüedad	Tokens	Interpretaciones posibles
5	Árbol visual	Nodo raíz	PNG con errores en rojo
6	Resultado	Lista de errores	Mensaje + sugerencia

El parser es el motor central del sistema: sin él no se pueden obtener los DAGs de los sintagmas nominales y por tanto no hay unificación posible. El DFA es prerequisite del parser y la detección de ambigüedad opera sobre los mismos tokens. Todas las herramientas están construidas desde cero en Python puro sin librerías de PLN externas.